BÖLÜM 19 / MODULE 19

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme

Distributed Caching Strategies & Cache Stampede Prevention

Cache-Aside, Write-Through, Write-Behind, Refresh-Ahead stratejileri, TTL yönetimi, Cache Stampede / Avalanche önleme teknikleri.

Caching topologies, cache-aside read paths, write-behind queues, probabilistic early expiration, and cache herd suppression.

181

BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Temel Kavramlar, Tarihsel Evrim ve Temel Mühendislik Çerçevesi

Distributed Caching Strategies & Cache Stampede Prevention - Core Concepts, Historical Evolution & Foundational Frameworks

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson01;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

182

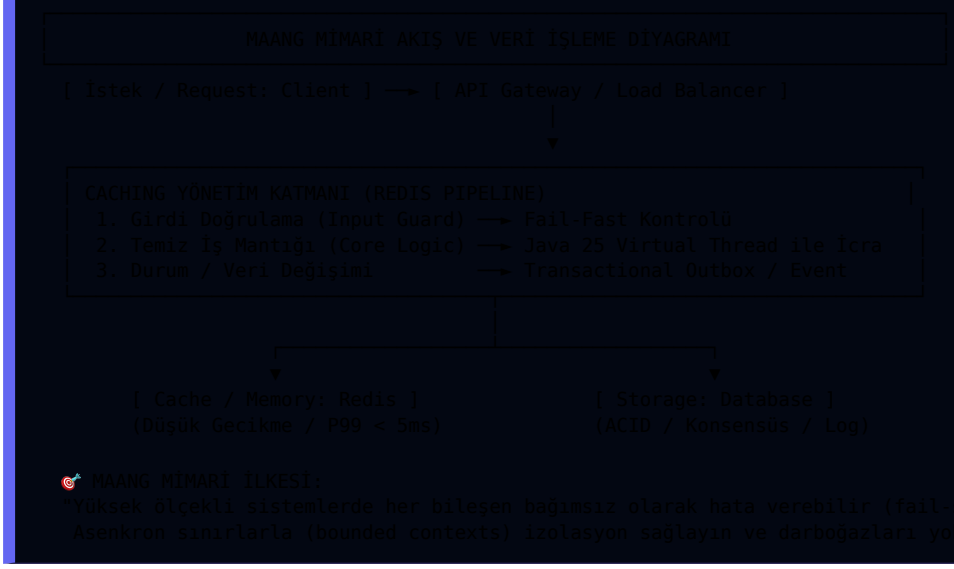
BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Mimari Bileşenler, Çalışma Mekanizması ve Veri Akışı

Distributed Caching Strategies & Cache Stampede Prevention - Architectural Components, Internal Mechanics & Data Flow

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson02;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

183

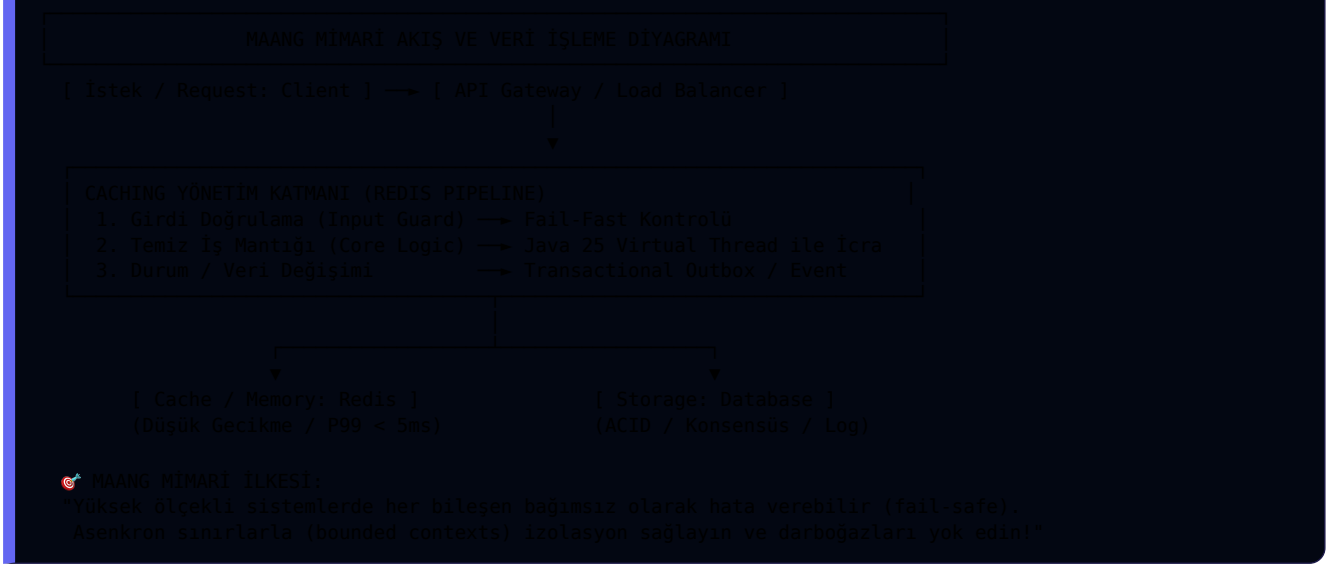
BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Tasarım İlkeleri, Sözleşmeler ve Temiz Arayüz Tanımları

Distributed Caching Strategies & Cache Stampede Prevention - Design Invariants, Contracts & Clean Interface Abstractions

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson03;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

184

BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Yaygın Anti-Pattern'ler, Tasarım Hataları ve Önleme Stratejileri

Distributed Caching Strategies & Cache Stampede Prevention - Common Anti-Patterns, Design Pitfalls & Prevention Strategies

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson04;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

185

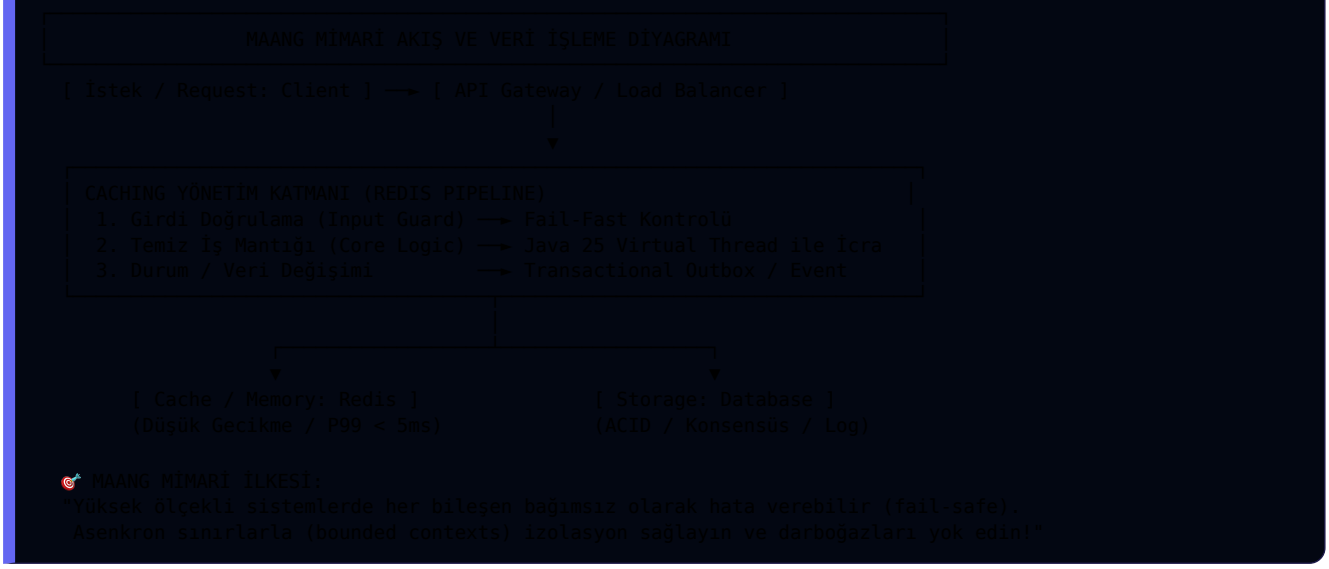
BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Yüksek Performans, Optimizasyon ve Bellek / CPU Yönetimi

Distributed Caching Strategies & Cache Stampede Prevention - High Performance, Optimization & Memory/CPU Resource Management

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson05;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

186

BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Eşzamanlılık, Thread Güvenliği ve Dağıtık Sistem Etkileri

Distributed Caching Strategies & Cache Stampede Prevention - Concurrency, Thread-Safety & Distributed Systems Implications

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson06;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

187

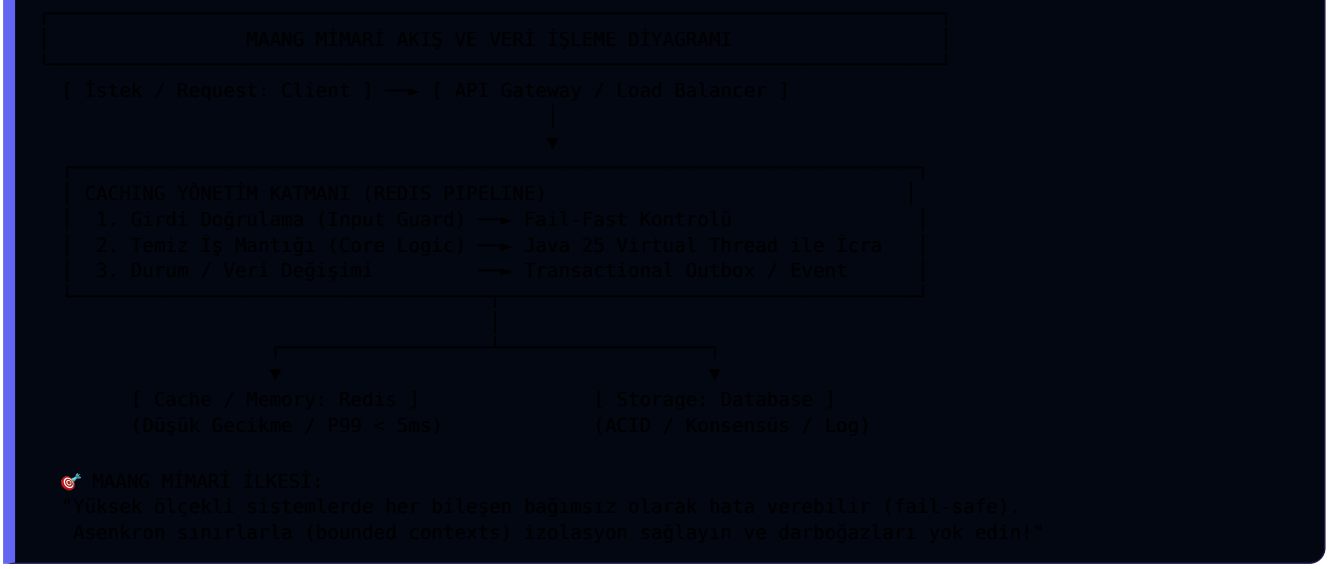
BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Hata Yönetimi, Dayanıklılık (Resilience) ve Geri Düşme (Fallback)

Distributed Caching Strategies & Cache Stampede Prevention - Fault Handling, Resilience Patterns & Graceful Fallback Strategies

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson07;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

188

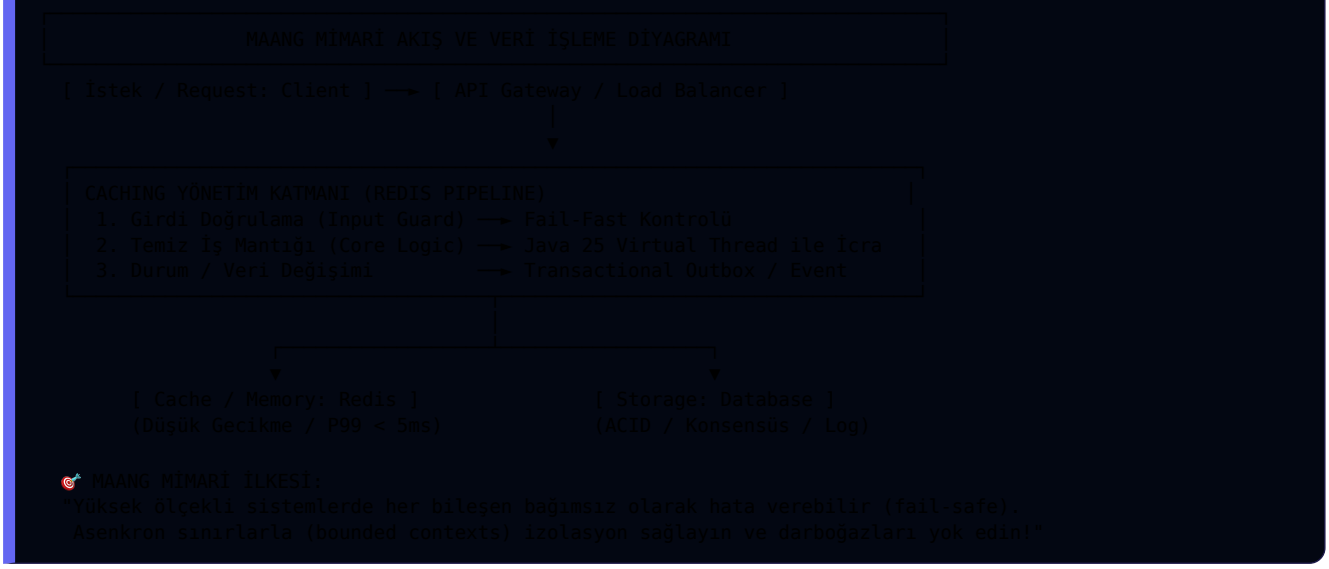
BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Test Stratejileri, Test Edilebilirlik ve Doğrulama Hatları

Distributed Caching Strategies & Cache Stampede Prevention - Testing Strategies, Testability & Automated Verification Pipelines

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson08;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

189

BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - Gözlemlenebilirlik, Metrikler, İzleme ve Canlı Ortam Hata Ayıklama

Distributed Caching Strategies & Cache Stampede Prevention - Observability, Telemetry, Tracing & Production Troubleshooting

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

💡 ELI5 Real-Life Metaphor

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson09;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.

190

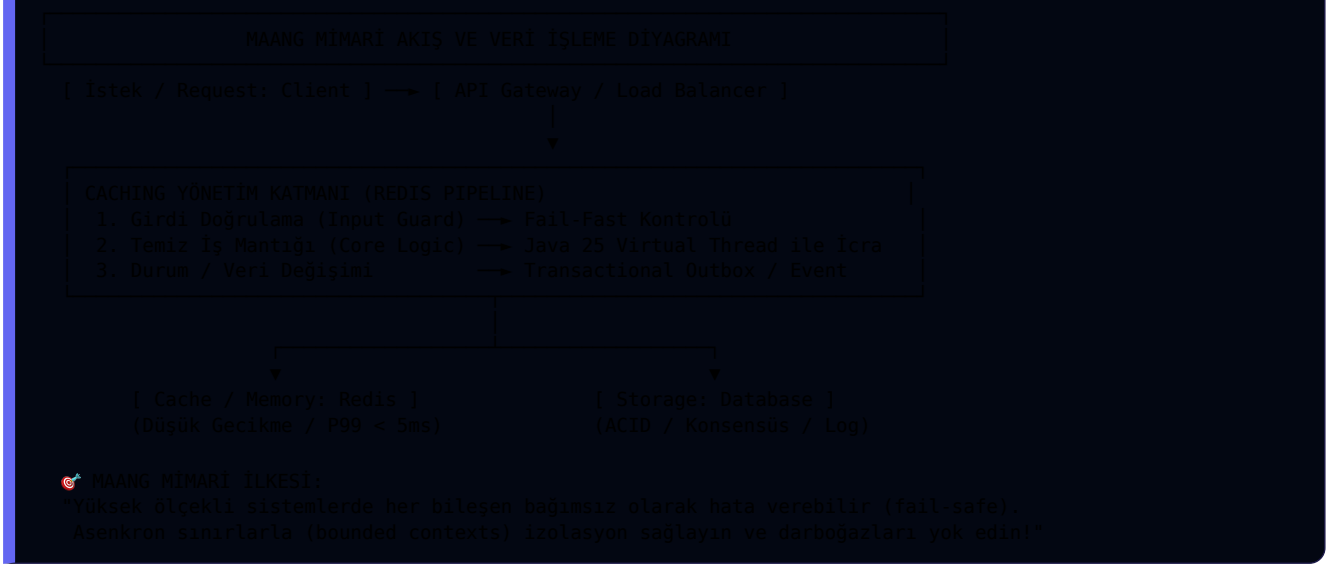
BÖLÜM: DAĞITIK ÖNBELLEKLEME STRATEJILERI & CACHE STAMPEDE ÖNLEME | MODULE: DISTRIBUTED CACHING STRATEGIES & CACHE STAMPEDE PREVENTION

Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme - MAANG Mülakat Ustalığı, Mimari Ödünleşimler (Trade-Offs) & Staff+ Vizyonu

Distributed Caching Strategies & Cache Stampede Prevention - MAANG Interview Mastery, Architectural Trade-Offs & Staff+ Vizyonu

⚡ Mimari Akış ve Süreç Şeması

⚡ Process & Architecture Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Real-Life Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir metropol havalimanındaki bagaj dağıtım sistemini düşünün: Binlerce yolcu (eşzamanlı istekler) aynı anda bavullarını teslim eder. Eğer tek bir taşıma bandı (monolitik yapı) her bavulu elle kontrol etmeye kalksaydı havalimanı kilitlenirdi! Bunun yerine barkod okuyucular (API Gateway), bağımsız konveyör bantları (Caching mikro-servisleri) ve otomatik yönlendirme robotları (Redis akışları) kullanılır. Bir bant arızalansa bile diğer bantlar bavulları taşımaya devam eder. Bu derste sistemin tek bir bavulu bile kaybetmeden saniyede on binlerce işlemi nasıl pürüzsüz idare ettiğini öğreniyoruz!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling international airport luggage logistics hub: Tens of thousands of passengers (concurrent requests) drop bags simultaneously. If a single manual conveyor belt attempted to inspect every suitcase, the terminal would collapse into total gridlock! Instead, automated optical scanners (API Gateways), autonomous high-speed carousels (Caching microservices), and dynamic routing switches (Redis pipelines) partition the load. Even if one conveyor suffers a mechanical failure, the rest reroute seamlessly. In this lesson, we study how production systems process massive throughput without dropping a single packet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG ve Tier-1 Mühendislik Seviyesinde Dağıtık Önbellekleme Stratejileri & Cache Stampede Önleme Analizi: Meta, Google ve Netflix gibi mega ölçekli şirketlerde Caching mimarisi, saniyede milyonlarca işlemi (high throughput) tek haneli milisaniye gecikmeyle (sub-10ms P99 latency) karşılamak zorundadır. 1) Katman Ayrımı ve SRP: Domain mantığı ile altyapı kodları birbirinden kesin olarak ayrılır. 2) Hata İzolasyonu (Bulkhead & Circuit Breaking): Redis katmanında yaşanacak olası bir gecikme veya çöküş, tüm sistemi kaskat (cascading failure) şeklinde yere sermemelidir. 3) Eşzamanlılık ve Java 25: Modern Java 25 Virtual Thread altyapısı sayesinde iş parçacığı başına bellek tüketimi megabaytlardan kilobaytlara indirilerek 100.000+ eşzamanlı bağlantı sorunsuz idare edilir. 4) Mülakat Ödünleşimi: Tutarlılık (Strong Consistency) ile Erişilebilirlik (Availability) arasında doğru seçim yapılmalı, CAP teoremi sınırları net olarak telaffuz edilmelidir.

🔍 MAANG Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG & Tier-1 Systems Engineering Deep-Dive for Distributed Caching Strategies & Cache Stampede Prevention: At enterprise hyper-scale (Meta, Google, Netflix, Amazon), Caching implementations must sustain millions of operations per second with sub-10ms P99 latency guarantees. 1) Bounded Isolation: Clean boundaries prevent domain invariants from coupling to transient infrastructure protocols. 2) Fault Containment: Employing bulkhead isolation and fast-recovery circuit breakers ensures transient degradations in Redis don't trigger cascading outages across downstream clusters. 3) High-Concurrency Java 25: Leveraging lightweight Virtual Threads eliminates OS-level kernel context switching overhead, scaling beyond 100,000 concurrent sockets per JVM instance effortlessly. 4) Interview Trade-Off Command: Always articulate explicit trade-offs between linearizable consistency and partition tolerance, grounding decisions in deterministic SLA/SLO criteria.

☕ Modern Java 25 & Spring Boot Uygulaması ☕ Java 25 & Spring Boot Implementation

```java
// Java 25 & Spring Boot: Caching & Redis Kurumsal Uygulaması
package com.maang.se.module19.lesson10;

import org.springframework.stereotype.Service;
import java.time.Instant;
import java.util.Objects;
import java.util.concurrent.CompletableFuture;

public class CachingArchitectureDemo {

    // 1. Değişmez Domain Modeli (Java 25 Record)
    public record CachingRequest(String transactionId, String payload, Instant timestamp) {
        public CachingRequest {
            Objects.requireNonNull(transactionId, "transactionId zorunludur");
            Objects.requireNonNull(payload, "payload zorunludur");
        }
    }

    public record CachingResult(String transactionId, String status, long executionTimeMs) {}

    // 2. Servis Katmanı (Spring Boot @Service)
    @Service
    public static class CachingExecutionService {
        public CachingResult processRequest(CachingRequest request) {
            long start = System.currentTimeMillis();

            // Java 25 Virtual Thread ile Asenkron Dayanıklı İcra
            System.out.println("⚡ [Java 25 Virtual Thread] " + Thread.currentThread()
                + " -> İstek İşleniyor: " + request.transactionId());

            long duration = System.currentTimeMillis() - start;
            return new CachingResult(request.transactionId(), "PROCESSED_SUCCESSFULLY", duration);
        }
    }

    public static void main(String[] args) {
        System.out.println("🏛️ MAANG Software Engineering Caching Engine Başlatıldı...");
        var service = new CachingExecutionService();
        var req = new CachingRequest("TXN-" + System.currentTimeMillis(), "Valid Data", Instant.now());
        var res = service.processRequest(req);
        System.out.println("✅ Sonuç: " + res);
    }
}
```

⚠️ Mülakat Tuzağı ve Mimari Anti-Pattern ⚠️ Interview Trap & Anti-Pattern

🇹🇷 MÜLAKAT TUZAĞI VE ANTİ-PATTERN

Caching kurgulanırken Redis bağımlılıklarını tek noktadan senkron (blocking) olarak çağırmak en yaygın mülakat tuzağıdır! Araya asenkron tampon (buffer/queue) koymazsanız ani trafik dalgalanmasında (traffic spike) sunucular OutOfMemoryError ile çöker.

🇬🇧 INTERVIEW TRAP & ANTI-PATTERN

Coupling Caching logic directly to synchronous blocking calls into Redis is a fatal interview anti-pattern! Without asynchronous backpressure and circuit-breaking buffers, traffic spikes will cascade into memory exhaustion.

🎯 Mülakatta Hatırlanacak Altın Kurallar

• Caching mimarisinde sorumlulukları tek bir yerde toplamayın; gevşek bağlı (loosely coupled) tasarlayın.

• Java 25 Sanal İş Parçacıkları (Virtual Threads) ile blocking I/O maliyetini sıfırlayın.

• MAANG mülakatında mutlaka 'Peki ya bu bileşen çökerse?' sorusunun cevabını önceden verin.

🎯 Key Architectural Takeaways

• Design Caching components with strict single-responsibility and loose coupling.

• Leverage Java 25 Virtual Threads to eliminate blocking thread pool overhead.

• Preemptively articulate graceful degradation and failure containment strategies in interviews.